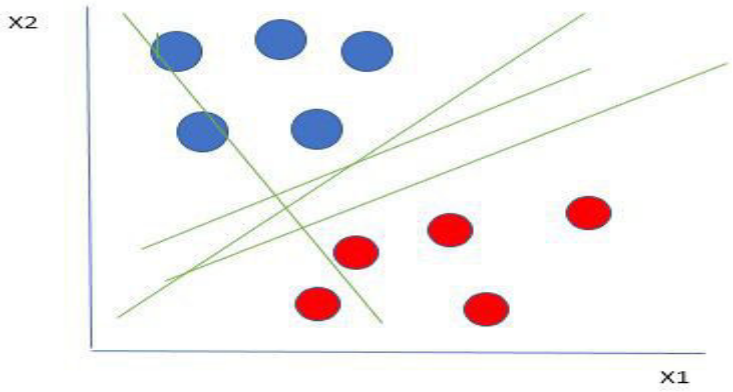
Support Vector Machine (SVM)

- Support Vector Machine (SVM) is a powerful machine learning algorithm used for linear or nonlinear classification, regression, and even outlier detection tasks.
- SVMs can be used for a variety of tasks, such as text classification, image classification, spam detection, handwriting identification, gene expression analysis, face detection, and anomaly detection.
- SVMs are adaptable and efficient in a variety of applications because they can manage high-dimensional data and nonlinear relationships.
- SVM algorithms are very effective as we try to find the maximum separating hyperplane between the different classes available in the target feature
- Support Vector Machine (SVM) is a supervised machine learning algorithm used for both classification and regression.
- Though we say regression problems as well it's best suited for classification.
- The main objective of the SVM algorithm is to find the optimal hyperplane in an N-dimensional space that can separate the data points in different classes in the feature space.
- The hyperplane tries that the margin between the closest points of different classes should be as maximum as possible.
- The dimension of the hyperplane depends upon the number of features.
- If the number of input features is two, then the hyperplane is just a line.
- If the number of input features is three, then the hyperplane becomes a 2-D plane.
- It becomes difficult to imagine when the number of features exceeds three.
- Let's consider two independent variables x_1, x_2, and one dependent variable which is either a blue circle or a red circle.

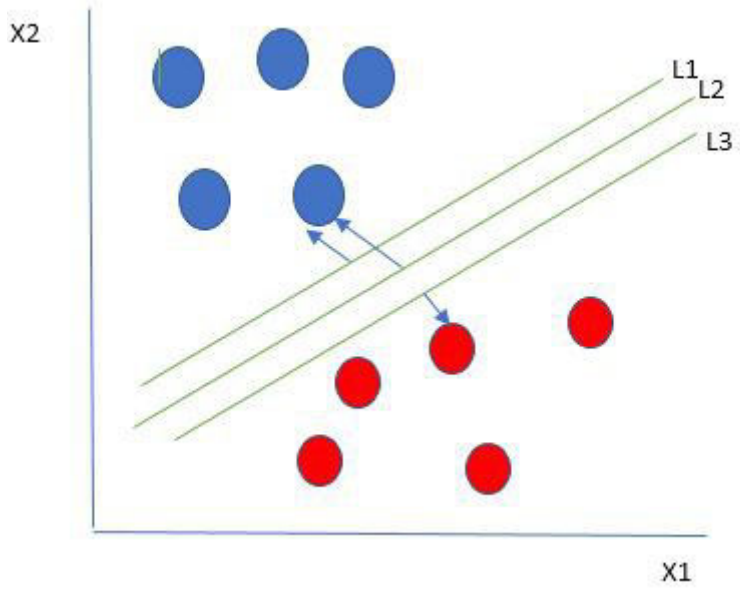


Linearly Separable Data points

From the figure above it's very clear that there are multiple lines (our hyperplane here is a line because we are considering only two input features x_1, x_2) that segregate our data points or do a classification between red and blue circles. So how do we choose the best line or in general the best hyperplane that segregates our data points?

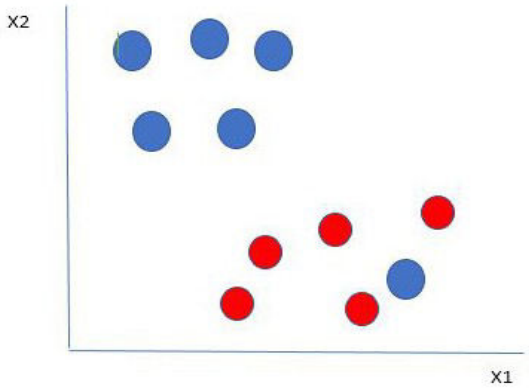
How does SVM work?

One reasonable choice as the best hyperplane is the one that represents the largest separation or margin between the two classes.



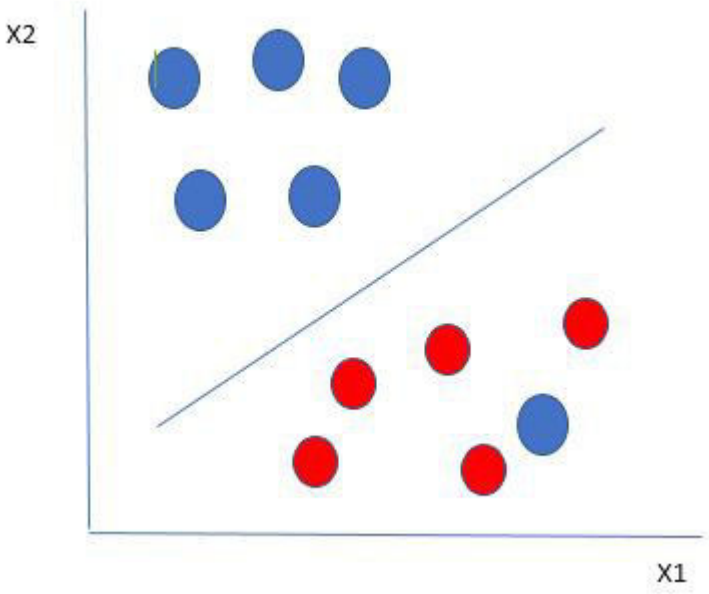
Multiple hyperplanes separate the data from two classes

- So we choose the hyperplane whose distance from it to the nearest data point on each side is maximized.
- If such a hyperplane exists it is known as the **maximum-margin hyperplane/hard margin**. So from the above figure, we choose L2. Let's consider a scenario like shown below



Selecting hyperplane for data with outlier

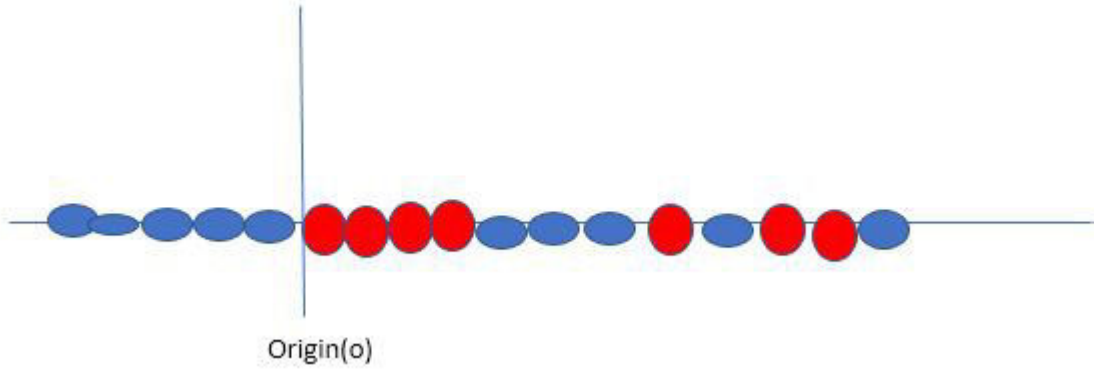
- Here we have one blue ball in the boundary of the red ball.
- So how does SVM classify the data? It's simple!
- The blue ball in the boundary of red ones is an outlier of blue balls.
- The SVM algorithm has the characteristics to ignore the outlier and finds the best hyperplane that maximizes the margin.
- SVM is robust to outliers.



Hyperplane which is the most optimized one

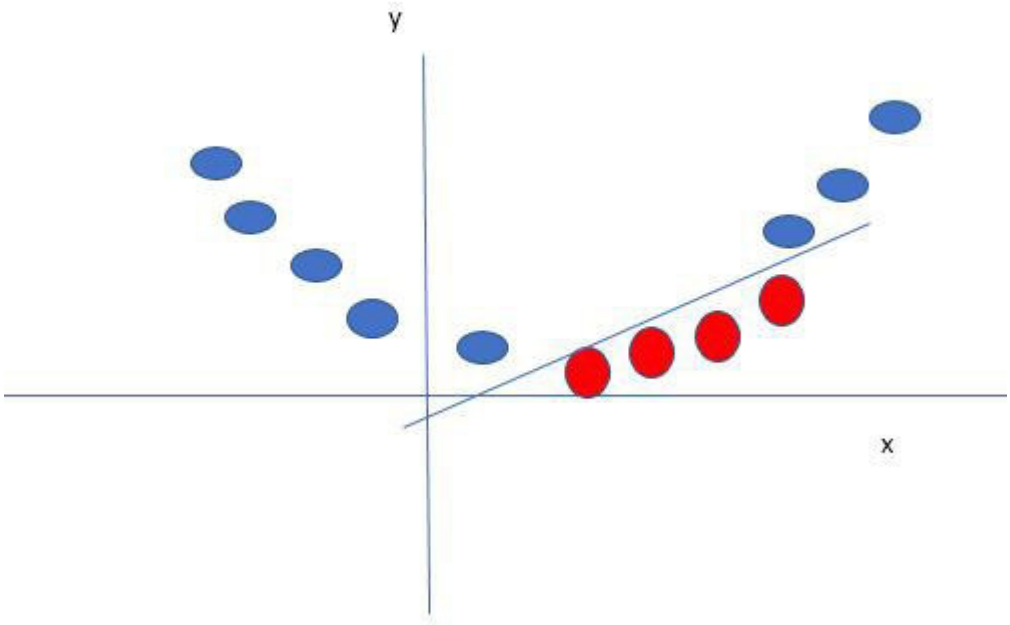
- So in this type of data point what SVM does is, finds the maximum margin as done with previous data sets along with that it adds a penalty each time a point crosses the margin.
- So the margins in these types of cases are called **soft margins**.
- When there is a soft margin to the data set, the SVM tries to minimize *(1/margin+∧(∑penalty))*.
- Hinge loss is a commonly used penalty.
- If no violations no hinge loss.
- If violations hinge loss proportional to the distance of violation.

Till now, we were talking about linearly separable data(the group of blue balls and red balls are separable by a straight line/linear line). What to do if data are not linearly separable?



Original 1D dataset for classification

Say, our data is shown in the figure above. SVM solves this by creating a new variable using a **kernel**. We call a point x_i on the line and we create a new variable y_i as a function of distance from origin o.so if we plot this we get something like as shown below



Mapping 1D data to 2D to become able to separate the two classes

In this case, the new variable y is created as a function of distance from the origin. A non-linear function that creates a new variable is referred to as a kernel.

Support Vector Machine Terminologies:

1. **Hyperplane:** Hyperplane is the decision boundary that is used to separate the data points of different classes in a feature space. In the case of linear classifications, it will be a linear equation i.e. wx+b = 0.

2. **Support Vectors:** Support vectors are the closest data points to the hyperplane, which makes a critical role in deciding the hyperplane and margin.

3. **Margin**: Margin is the distance between the support vector and hyperplane. The main objective of the support vector machine algorithm is to maximize the margin. The wider margin indicates better classification performance.

4. **Kernel**: Kernel is the mathematical function, which is used in SVM to map the original input data points into high-dimensional feature spaces, so, that the hyperplane can be easily found out even if the data points are not linearly separable in the original input space. Some of the common kernel functions are linear, polynomial, radial basis function(RBF), and sigmoid.

5. **Hard Margin:** The maximum-margin hyperplane or the hard margin hyperplane is a hyperplane that properly separates the data points of different categories without any misclassifications.

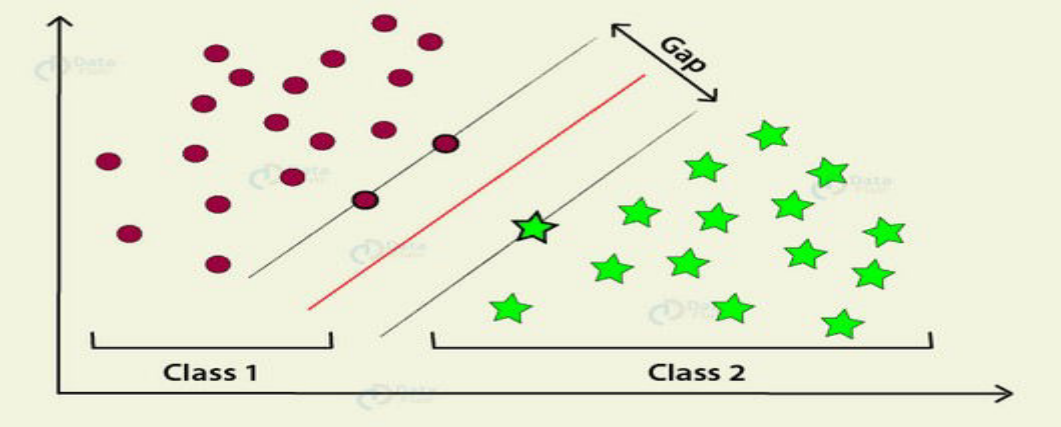
6. **Soft Margin:** When the data is not perfectly separable or contains outliers, SVM permits a soft margin technique. Each data point has a slack variable introduced by the soft-margin SVM formulation, which softens the strict margin requirement and permits certain misclassifications or violations. It discovers a compromise between increasing the margin and reducing violations.

7. **C:** Margin maximisation and misclassification fines are balanced by the regularisation parameter C in SVM. The penalty for going over the margin or misclassifying data items is decided by it. A stricter penalty is imposed with a greater value of C, which results in a smaller margin and perhaps fewer misclassifications.

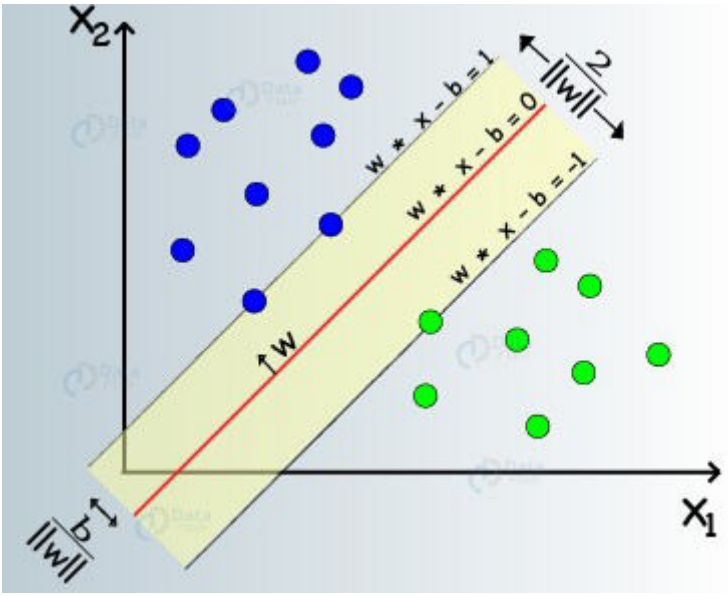
8. **Hinge Loss:** A typical loss function in SVMs is hinge loss. It punishes incorrect classifications or margin violations. The objective function in SVM is frequently formed by combining it with the regularisation term.

9. **Dual Problem:** A dual Problem of the optimisation problem that requires locating the Lagrange multipliers related to the support vectors can be used to solve SVM. The dual formulation enables the use of kernel tricks and more effective computing.

What is SVM?



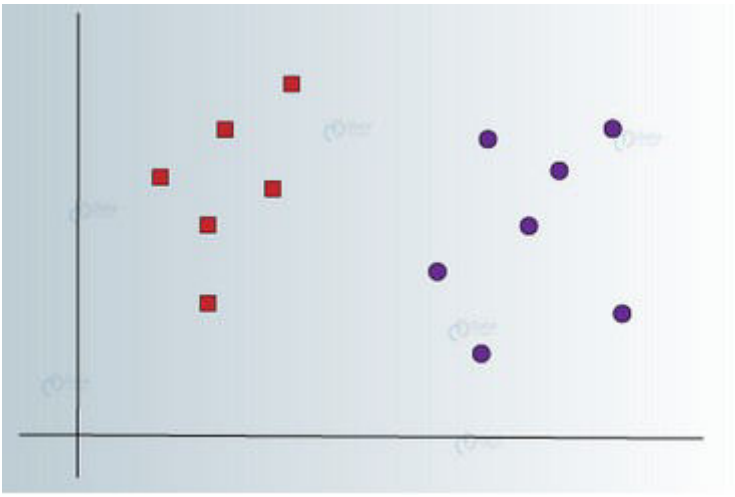
- Support Vector Machines are a type of supervised machine learning algorithm that provides analysis of data for classification and regression analysis.
- While they can be used for regression, SVM is mostly used for classification.
- We carry out plotting in the n-dimensional space.
- Value of each feature is also the value of the specific coordinate.
- Then, we find the ideal hyperplane that differentiates between the two classes.
- These support vectors are the coordinate representations of individual observation. It is a frontier method for segregating the two classes.



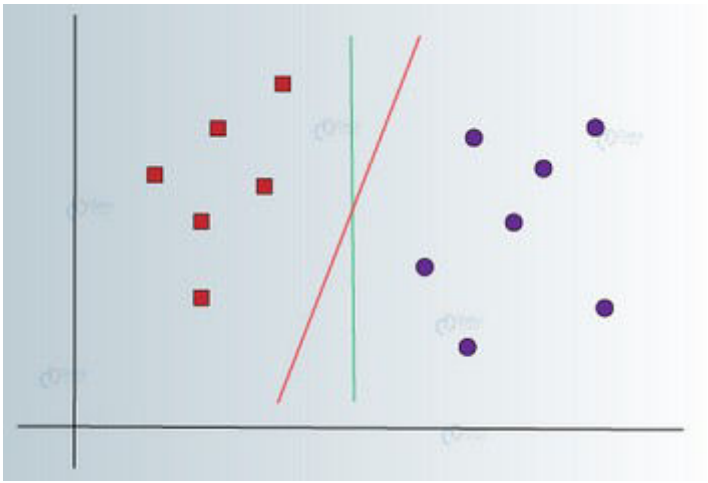
How does SVM work?

- The basic principle behind the working of Support vector machines is simple – Create a hyperplane that separates the dataset into classes.

- Let us start with a sample problem. Suppose that for a given dataset, you have to classify red triangles from blue circles.
- Your goal is to create a line that classifies the data into two classes, creating a distinction between red triangles and blue circles.



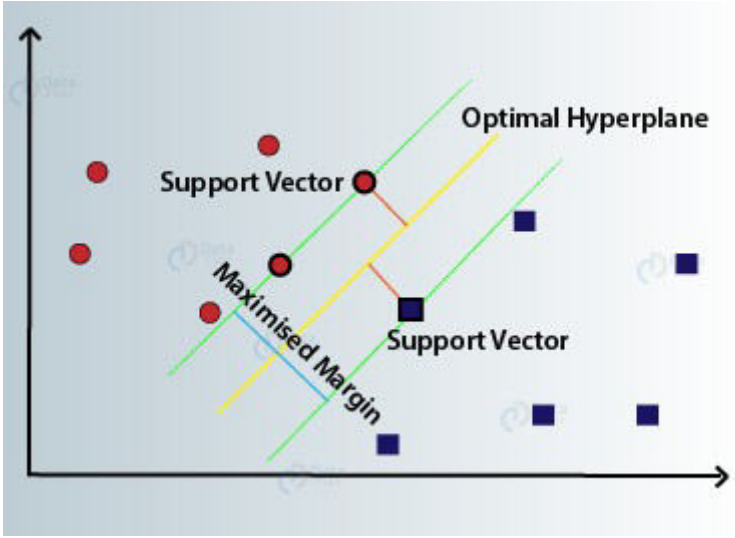
While one can hypothesize a clear line that separates the two classes, there can be many lines that can do this job. Therefore, there is not a single line that you can agree on which can perform this task. Let us visualize some of the lines that can differentiate between the two classes as follows –



In the above visualizations, we have a green line and a red line. Which one do you think would better differentiate the data into two classes? If you choose the red line, then it is the ideal line that partitions the two classes properly. However, we still have not concretized the fact that it is the universal line that would classify our data most efficiently.

The green line cannot be the ideal line as it lies too close to the red class. Therefore, it does not provide a proper generalization which is our end goal.

According to SVM, we have to find the points that lie closest to both the classes. These points are known as support vectors. In the next step, we find the proximity between our dividing plane and the support vectors. The distance between the points and the dividing line is known as margin. The aim of an SVM algorithm is to maximize this very margin. When the margin reaches its maximum, the hyperplane becomes the optimal one.



The SVM model tries to enlarge the distance between the two classes by creating a well-defined decision boundary. In the above case, our hyperplane divided the data. While our data was in 2 dimensions, the hyperplane was of 1 dimension. For higher dimensions, say, an n-dimensional Euclidean Space, we have an n-1 dimensional subset that divides the space into two disconnected components.

Mathematical intuition of Support Vector Machine

- Consider a binary classification problem with two classes, labeled as +1 and -1.
- We have a training dataset consisting of input feature vectors X and their corresponding class labels Y.
- The equation for the linear hyperplane can be written as:

$$w^T x + b = 0$$

- The vector W represents the normal vector to the hyperplane. i.e the direction perpendicular to the hyperplane.
- The parameter **b** in the equation represents the offset or distance of the hyperplane from the origin along the normal vector **w**.
- The distance between a data point x_i and the decision boundary can be calculated as:

$$d_i = \frac{w^T x_i + b}{||w||}$$

where ||w|| represents the Euclidean norm of the weight vector w. Euclidean norm of the normal vector W

For Linear SVM classifier :

$$\hat{y} = \begin{cases} 1 & : \ w^T x + b \geq 0 \\ 0 & : \ w^T x + b < 0 \end{cases}$$

Optimization:

- **For Hard margin linear SVM classifier:**

$$\underset{w,b}{\text{minimize}} \frac{1}{2} w^T w = \underset{W,b}{\text{minimize}} \frac{1}{2} \|w\|^2$$
$$\text{subject to } y_i(w^T x_i + b) \geq 1 \ for \ i = 1, 2, 3, \cdots, m$$

- The target variable or label for the ith training instance is denoted by the symbol t_i in this statement.
- And t_i=-1 for negative occurrences (when y_i= 0) and t_i=1positive instances (when y_i = 1) respectively.
- Because we require the decision boundary that satisfy the constraint:

$$t_i(w^T x_i + b) \geq 1$$

- **For Soft margin linear SVM classifier:**

$$\underset{w,b}{\text{minimize}}\ \frac{1}{2}w^T w + C\sum_{i=1}^{m}\zeta_i$$

$$\text{subject to } y_i(w^T x_i + b) \geq 1 - \zeta_i \ \ and\ \zeta_i \geq 0 \ \ for\ i = 1, 2, 3, \cdots, m$$

Dual Problem:

- A dual Problem of the optimisation problem that requires locating the Lagrange multipliers related to the support vectors can be used to solve SVM.
- The optimal Lagrange multipliers α(i) that maximize the following dual objective function

$$\underset{\alpha}{\text{maximize}} : \frac{1}{2} \sum_{i \to m} \sum_{j \to m} \alpha_i \alpha_j t_i t_j K(x_i, x_j) - \sum_{i \to m} \alpha_i$$

where,

- α_i is the Lagrange multiplier associated with the ith training sample.
- $K(x_i, x_j)$ is the kernel function that computes the similarity between two samples x_i and x_j. It allows SVM to handle nonlinear classification problems by implicitly mapping the samples into a higher-dimensional feature space.
- The term $\sum \alpha_i$ represents the sum of all Lagrange multipliers.

- The SVM decision boundary can be described in terms of these optimal Lagrange multipliers and the support vectors once the dual issue has been solved and the optimal Lagrange multipliers have been discovered.
- The training samples that have i > 0 are the support vectors, while the decision boundary is supplied by:

$$w = \sum_{i \to m} \alpha_i t_i K(x_i, x) + b$$
$$t_i(w^T x_i - b) = 1 \iff b = w^T x_i - t_i$$

Types of Support Vector Machine:

Based on the nature of the decision boundary, Support Vector Machines (SVM) can be divided into two main parts:

Linear SVM:

- Linear SVMs use a linear decision boundary to separate the data points of different classes.
- When the data can be precisely linearly separated, linear SVMs are very suitable.
- This means that a single straight line (in 2D) or a hyperplane (in higher dimensions) can entirely divide the data points into their respective classes.
- A hyperplane that maximizes the margin between the classes is the decision boundary.

Non-Linear SVM:

- Non-Linear SVM can be used to classify data when it cannot be separated into two classes by a straight line (in the case of 2D).
- By using kernel functions, nonlinear SVMs can handle nonlinearly separable data.
- The original input data is transformed by these kernel functions into a higher-dimensional feature space, where the data points can be linearly separated.
- A linear SVM is used to locate a nonlinear decision boundary in this modified space.

How do Kernels Learn Non-Linear Functions?

- Kernel machines are a class of pattern-analysis algorithms, the most well-known member of which is the support vector machine (SVM).
- Many classical classification algorithms work straightforward when the data is linearly separable.
- But when this algorithm has to work on the data that is not linearly separable, it has to use a different strategy.
- Kernels are used by classification algorithms to solve non-linear classification problems.
- We mostly refer to the kernel used in the Support Vector Machine (SVM) algorithm.

Need of the Kernel:

Let's say we have a 2-dimensional dataset with two classes of observations, as illustrated in fig-1 below, we need to design a function to separate them. As it can be seen below that the given data in a 2-dimensional space is not linearly separable.

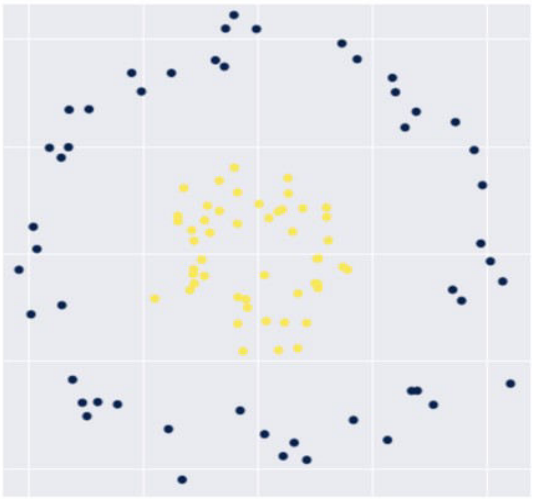


Fig-1

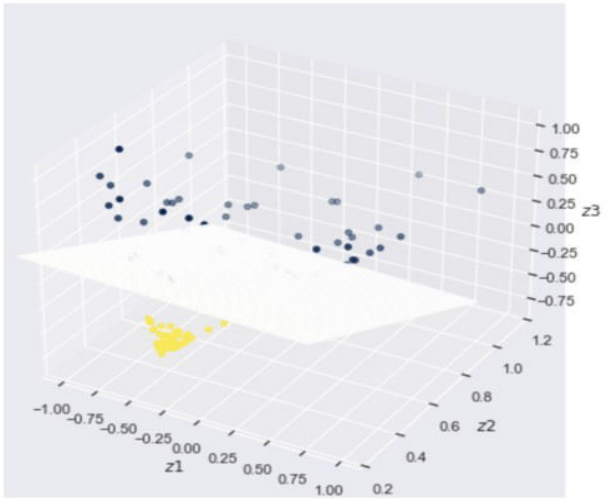


Fig-2

To segregate the data, we'd have to fit a function, which complicates our categorization challenge. But what if we could translate this data into a higher-dimensional (3D) space where a linear classifier could separate the data?

We can find a mapping from 2D to 3D space that allows us to linearly separate our observations, transform our two-dimensional data into three-dimensional data, help to find a linear decision boundary, fit a linear classifier (a plane dividing the data shown in fig-2) in 3D space.

This is when the kernel technique comes in handy. The goal is to use a function that projects / transform the data in such a way that it can be separated linearly. By using the maximum margin in the SVM method, data becomes linearly separable.

How Does a Kernel Work?

- Kernel machines are a class of pattern-analysis algorithms, the most well-known member of which is the support vector machine (SVM).
- The general objective of pattern analysis is to discover and investigate various sorts of relationships (for example, clusters, ranks, principal components, correlations, and classifications) in datasets.
- Many algorithms that solve these tasks require the data in raw representation to be explicitly transformed into feature vector representations via a user-specified feature map: kernel methods, on the other hand, require only a user-specified kernel, i.e., a similarity function over pairs of raw data points.
- Kernel methods are approaches for dealing with linearly inseparable data or non-linear data sets like those presented in fig-1.
- The concept is to use a mapping function to project nonlinear combinations of the original features onto a higher-dimensional space, where the data becomes linearly separable.
- The two-dimensional dataset (X1, X2) is projected into a new three-dimensional feature space (Z1, Z2, Z3) in the diagram above, where the classes become separable.

To grasp it completely, Assume we have two vectors, x and x*, in a 2D space (illustrated in fig-1) and want to find a linear classifier by performing a dot product between them. Unfortunately, in our current 2D vector space, the data is not linearly separable. We can address this challenge by mapping the two vectors to a 3D space.

$x \rightarrow \phi(x)$

$x^* \rightarrow \phi(x^*)$

Where (x) and (x*) are 3D representations of x and x*, respectively. Now we can discover our linear classifier in 3D space using the dot product of (x) and (x*) and then map back to 2D space using the dot product of (x) and (x*) like below.

$x^T x^* \rightarrow \phi(x)^T \phi(x*)$

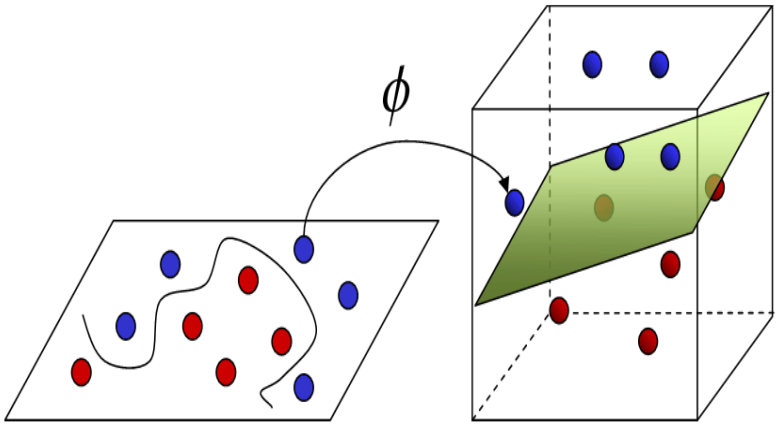
- A mapping function can be used to convert the training data into a higher-dimensional feature space, and then a linear SVM model can be trained to classify the data in this new feature space following the method outlined above.
- Using the mapping function, the new data may then be fed into the model for categorization. However, this method is computationally intensive. So, what is the solution?
- The approach is to use a method to avoid explicitly mapping the input data to a high-dimensional feature space in order to train linear learning algorithms to learn a nonlinear function or decision boundary.
- This is known as a kernel trick.

- It should be noted that the kernel trick is significantly more general than SVM.

The Kernel Trick

We've seen how higher-dimensional transformations can help us separate data so that classification predictions can be made. It appears that we will have to operate on the higher dimensional vectors in the modified feature space in order to train a support vector classifier and maximize our objective function.

In real-world applications, data may contain numerous features, and transformations using multiple polynomial combinations of these features will result in extremely large and prohibitive processing costs.



Kernel Trick

This problem can be solved using the kernel trick. Instead of explicitly applying the transformations (x) and representing the data by these transformed coordinates in the higher dimensional feature space, kernel methods represent the data only through a set of pairwise similarity comparisons between the original data observations x (with the original coordinates in the lower dimensional space).

Our kernel function takes in lower-dimensional inputs and outputs the dot product of converted vectors in higher-dimensional space. Other theorems guarantee that such kernel functions exist under certain conditions.

If a function can be written as an inner product of a mapping function, we only need to know that function, not the mapping function, as shown above. A Kernel Function is a name for the function.

SVM Kernel Functions:

SVM algorithms use a set of mathematical functions that are defined as the kernel. The function of kernel is to take data as input and transform it into the required form. Different SVM algorithms use different types of kernel functions. These functions can be different types. For example ***linear, nonlinear, polynomial, radial basis function (RBF), and sigmoid.***

Introduce Kernel functions for sequence data, graphs, text, images, as well as vectors. The most used type of kernel function is **RBF.** Because it has localized and finite response along the entire x-axis.

The kernel functions return the inner product between two points in a suitable feature space. Thus by defining a notion of similarity, with little computational cost even in very high-dimensional spaces.

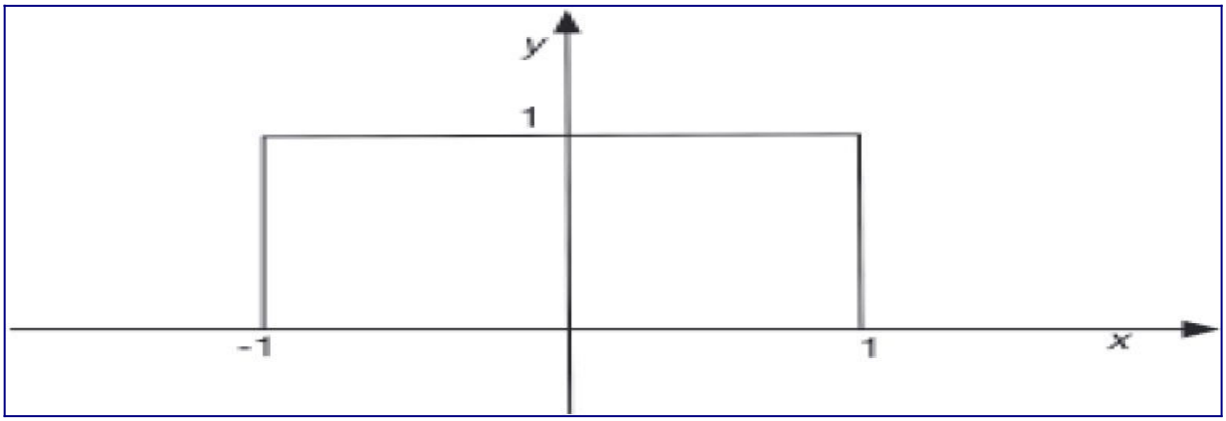
Kernel Rules

Define kernel or a window function as follows:

$$K\left(\overline{x}\right) = \begin{matrix} 1 & \text{if} \left\|\overline{x}\right\| \leq 1 \\ 0 & \text{otherwise} \end{matrix}$$

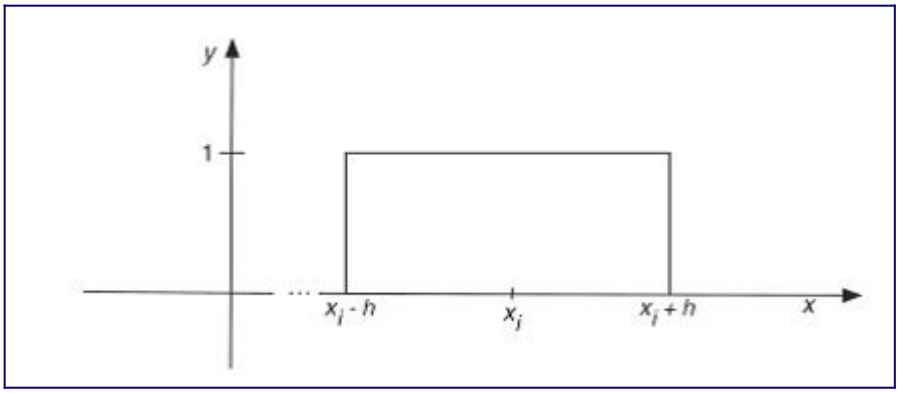
Kernel or a window function

This value of this function is 1 inside the closed ball of radius 1 centered at the origin, and 0 otherwise . As shown in the figure below:



Kernel or a window function

For a fixed xi, the function is K(z-xi)/h) = 1 inside the closed ball of radius h centered at xi, and 0 otherwise as shown in the figure below:



Kernel or a window function

So, by choosing the argument of K(·), you have moved the window to be centered at the point xi and to be of radius h.

DIFFERENT TYPES OF SVM KERNELS:

1. Polynomial kernel

It is popular in image processing.
Equation is:

$$k(\mathbf{x_i}, \mathbf{x_j}) = (\mathbf{x_i} \cdot \mathbf{x_j} + 1)^d$$

where d is the degree of the polynomial.

2. Gaussian kernel

It is a general-purpose kernel; used when there is no prior knowledge about the data. Equation is:

$$k(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)$$

3. Gaussian radial basis function (RBF)

It is a general-purpose kernel; used when there is no prior knowledge about the data. Equation is:

$$k(\mathbf{x_i}, \mathbf{x_j}) = \exp(-\gamma \|\mathbf{x_i} - \mathbf{x_j}\|^2)$$

for:

$$\gamma > 0$$

Gaussian radial basis function (RBF)

Sometimes parametrized using:

$$\gamma = 1/2\sigma^2$$

Gaussian radial basis function (RBF)

4. Laplace RBF kernel

It is general-purpose kernel; used when there is no prior knowledge about the data. Equation is:

$$k(x, y) = \exp\left(-\frac{||x - y||}{\sigma}\right)$$

5. Hyperbolic tangent kernel

We can use it in neural networks.
Equation is:

$$k(\mathbf{x_i}, \mathbf{x_j}) = \tanh(\kappa \mathbf{x_i} \cdot \mathbf{x_j} + c)$$

for some (not every) k>0 and c<0.

6. Sigmoid kernel

We can use it as the proxy for neural networks. Equation is

$$k(x, y) = \tanh(\alpha x^T y + c)$$

7. Bessel function of the first kind Kernel

We can use it to remove the cross term in mathematical functions. Equation is :

$$k(x, y) = \frac{J_{v+1}(\sigma||x - y||)}{||x - y||^{-n(v+1)}}$$

Equation of Bessel function of the first kind kernel

where j is the Bessel function of first kind.

8. ANOVA radial basis kernel

We can use it in regression problems. Equation is:

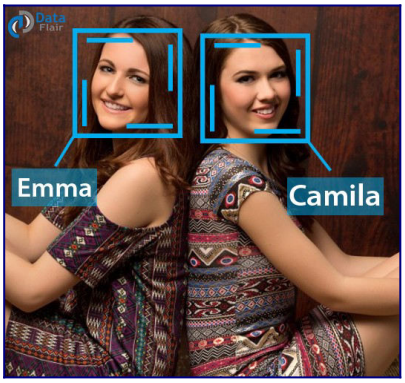
$$k(x, y) = \sum_{k=1}^{n} \exp(-\sigma(x^k - y^k)^2)^d$$

Applications of SVM:

Some of the areas where Support Vector Machines are used are as follows –

1.Face Detection

SVMs are capable of classifying images of persons in an environment by creating a square box that separates the face from the rest.



2.Text and hypertext categorization

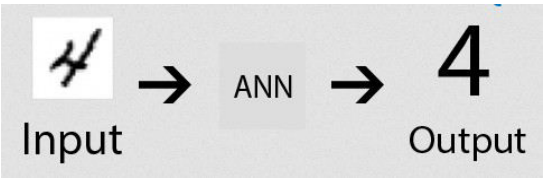
SVMs can be used for document classification in the sense that it performs the text and hypertext categorization. Based on the score generated, it performs a comparison with the threshold value.

3. Bioinformatics

In the field of bioinformatics, SVMs are used for protein and genomic classification. They can classify the genetic structure of the patients based on their biological problems.

4. Handwriting recognition

Another area where support vector machines are used for visual recognition is handwriting recognition.



Advantages of SVM

- Effective in high-dimensional cases.
- Its memory is efficient as it uses a subset of training points in the decision function called support vectors.
- Different kernel functions can be specified for the decision functions and its possible to specify custom kernels.

SVM implementation in Python

Predict if cancer is Benign or malignant. Using historical data about patients diagnosed with cancer enables doctors to differentiate malignant cases and benign ones are given independent attributes.

Steps

- Load the breast cancer dataset from sklearn.datasets
- Separate input features and target variables.
- Build and train the SVM classifiers using RBF kernel.
- Plot the scatter plot of the input features.
- Plot the decision boundary.
- Plot the decision boundary

Python Program:

```
# Load the important packages
from sklearn.datasets import load_breast_cancer
import matplotlib.pyplot as plt
from sklearn.inspection import DecisionBoundaryDisplay
from sklearn.svm import SVC

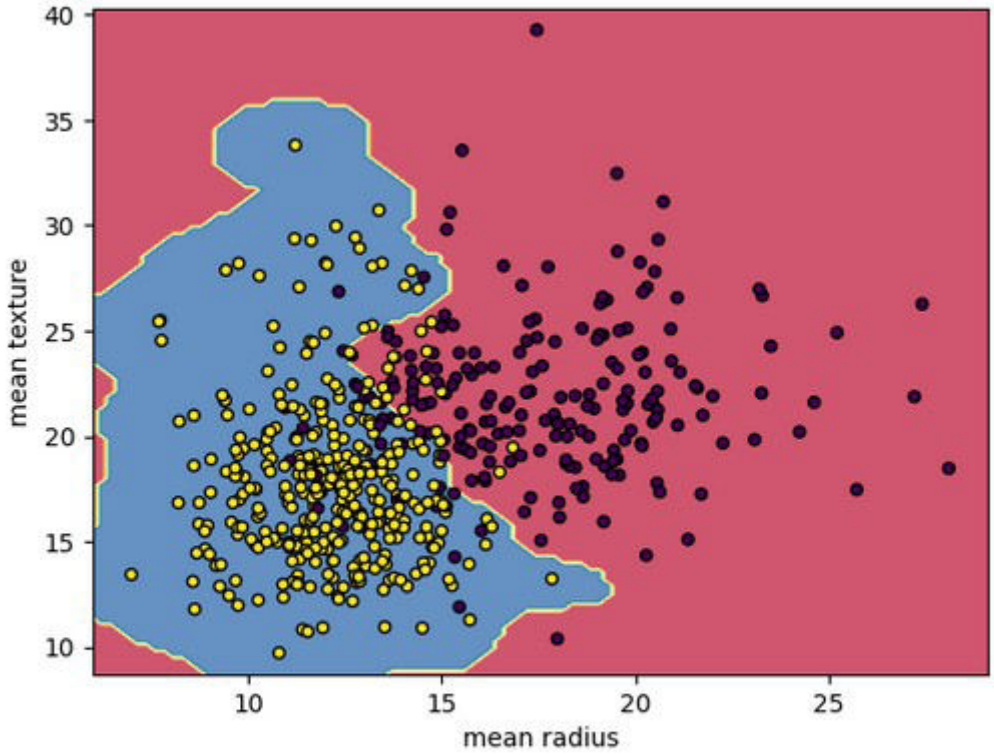
# Load the datasets
cancer = load_breast_cancer()
X = cancer.data[:, :2]
y = cancer.target

#Build the model
svm = SVC(kernel="rbf", gamma=0.5, C=1.0)
# Trained the model
svm.fit(X, y)

# Plot Decision Boundary
DecisionBoundaryDisplay.from estimator(
        svm,
        X,
        response method="predict",
        cmap=plt.cm.Spectral,
        alpha=0.8,
        xlabel=cancer.feature_names[0],
        ylabel=cancer.feature names[1],
    )

# Scatter plot
plt.scatter(X[:, 0], X[:, 1],
            c=y,
            s=20, edgecolors="k")
plt.show()
```

OUTPUT:



Breast Cancer Classifications with SVM RBF kernel